1.3 Programming Hints

It is not our purpose in this book to teach you how to program, only how to program better. We assume you are familiar with such fundamental concepts as variables, conditional statements (e.g., `if-then-else`, `case`), iteration primitives (e.g., `for-do`, `while-do`, `repeat-until`), subroutines, and functions. If you are unfamiliar with these concepts you may have picked up the wrong book, but buy it anyway for later use.

It is important to realize how much power there is in what you already know. In principle, every interesting algorithm/program can be built from what you learn in a first programming course. The powerful features of modern programming languages are not really necessary to build interesting things – only to do them in cleaner, better ways.

To put it another way, one becomes a good writer not by learning additional vocabulary words but by finding something to say. After one or two programming courses, you know all the words you need to make yourself understood. The problems in this book strive to give you something interesting to say.

We offer a few low-level coding hints that are helpful in building quality programs. The bad examples all come from actual submissions to the robot judge.

- *Write the Comments First* — Start your programs and procedures by writing a few sentences explaining what they are supposed to do. This is important because if you can't *easily* write these comments, you probably don't really understand what the program does. We find it much easier to debug our comments than our programs, and believe the additional typing is time very well spent. Of course, with the time pressure of a contest comes a tendency to get sloppy, but do so at your own risk.

- *Document Each Variable* — Write a one-line comment for each variable when you declare it so you know what it does. Again, if you can't describe it easily, you don't know why it is there. You will likely be living with the program for at least a few debug cycles, and this is a modest investment in readability which you will come to appreciate.

- *Use Symbolic Constants* — Whenever you have a constant in your program (input size, mathematical constant, data structure size, etc.) declare it to be so at the top of your program. Horribly insidious errors can result from using inconsistent constants in a program. Of course, the symbolic name helps only if you actually use it in your program whenever you need the constant...

- *Use Enumerated Types for a Reason* — Enumerated types (i.e., symbolic variables such Booleans (`true`,`false`)) can be terrific aids to understanding. However, they are often unnecessary in short programs. Note this example representing the suit (club, diamond, heart, spade) of a deck of cards:

```
switch(cursuit) {
    case 'C':
      newcard.suit = C;
```

```
      break;
    case 'D':
      newcard.suit = D;
      break;
    case 'H':
      newcard.suit = H;
      break;
    case 'S':
      newcard.suit = S;
...
```

No additional clarity arises from using the enumerated variables `(C,D,H,S)` over the original character representation `('C','D','H','S')`, only additional opportunities for error.

- *Use Subroutines To Avoid Redundant Code* — Read the following program fragment managing the state of a rectangular board, and think how you might shorten and simplify it:

```
...

     while (c != '0') {
        scanf("%c", &c);
        if (c == 'A') {
            if (row-1 >= 0) {
                temp = b[row-1][col];
                b[row-1][col] = ' ';
                b[row][col] = temp;
                row = row-1;
            }
        }
        else if (c == 'B') {
            if (row+1 <= BOARDSIZE-1) {
                temp = b[row+1][col];
                b[row+1][col] = ' ';
                b[row][col] = temp;
                row = row+1;
            }
        }
...
```

In the full program, there were four blocks of three lines each moving a value to a neighboring cell. Mistyping a single + or − would have lethal consequences. Much safer would be to write a single move-swap routine and call it with the proper arguments.

- *Make Your Debugging Statements Meaningful* — Learn to use the debugging environment on your system. This will enable you to stop execution at a given statement or condition, so you can see what the values of all associated variables are. This is usually faster and easier than typing in a bunch of print statements. But if you *are* going to insert debugging print statements, make them say something. Print out all relevant variables, and *label* the printed quantity with the variable name. Otherwise it is easy to get lost in your own debugging output.

Most computer science students are now well-versed in *object-oriented programming*, a software engineering philosophy designed to construct reusable software components and exploit them. Object-oriented programming is useful to build large, reusable programs. However, most of the programming challenge problems in this book are designed to be solved by short, clever programs. The basic assumption of object-oriented programming just does not apply in this domain, so defining complicated new objects (as opposed to *using* predefined objects) is likely to be a waste of time.

The trick to successful programming is not abandoning style, but using one appropriate to the scale of the job.

1.4 Elementary Data Types

Simple data structures such as arrays have an important advantage over more sophisticated data structures such as linked lists: they are simple. Many kinds of errors in pointer-based structures simply cannot happen with static arrays.

The sign of a mature professional is keeping the simple jobs simple. This is particularly challenging for those who are just learning a new subject. Medical students provide a classic example of this problem. After sitting through a few lectures on obscure tropical diseases, a young doctor worries that any patient with a sniffle and a rash might have the Ebola virus or bubonic plague, while a more experienced physician just sends them home with a bottle of aspirin.

Likewise, you may have recently learned about balanced binary search trees, exception handling, parallel processing, and various models of object inheritance. These are all useful and important subjects. But they are not necessarily the best way to get a correct program working for a simple job.

So, yes, pointer-based structures are very powerful if you do not know the maximum size of the problem in advance, or in supporting fast search and update operations. However, many of the problems you will be solving here have maximum sizes specified. Further, the robot judge typically allows several seconds for your job to complete, which is a lot of computation time when you stop to think about it. You don't get extra points for finishing faster.

So what is the simple, mature approach to data structures? First, be familiar with the basic primitive data *types* built into your programming language. In principle, you can build just about anything you want from just these:

- *Arrays* — This workhorse data type permits access to data by position, not content, just like the house numbers on a street permit access by address, not

name. They are used to store sequences of single-type elements such as integers, reals, or compound objects such as records. Arrays of characters can be used to represent text strings, while arrays of text strings can be used to represent just about anything.

Sentinels can be a useful technique to simplify array-based programming. A sentinel is a guard element, implicitly checking that the program does not run beyond the bounds of the array without performing an explicit test. Consider the case of inserting element x into the proper position among n elements in a sorted array a. We can explicitly test each step to see whether we have hit the bottom of the array as on the left:

```
i = n;
while ((a[i]>=x) && (i>=1)) {
        a[i] = a[i-1];
        i=i-1;
}
a[i+1] = x;
```

```
i = n;
a[0] = - MAXINT;
while (a[i] >= x) {
        a[i] = a[i-1];
        i=i-1;
}
a[i+1] = x;
```

or, we can make sure that fake element `a[0]` is smaller than anything it will encounter as on the right. Proper use of sentinels, and making sure that your array is a little larger than it presumably needs to be, can help avoid many boundary errors.

- *Multidimensional Arrays* — Rectangular grid structures such as chessboards and images comes to mind first when thinking about two-dimensional arrays, but more generally they can be used to group together *homogeneous* data records. For example, an array of n points in the $x - y$ plane can be thought of as an $n \times 2$ array, where the second argument (0 or 1) of $A[i][j]$ dictates whether we are referring to the x or y coordinate of the point.

- *Records* — These are used to group together *heterogeneous* data records. For example, an array of people-records can lump together people's names, ID numbers, heights, and weights into a simple package. Records are important for conceptual clarity in large programs, but such fields can often be harmlessly represented using separate arrays in programs of modest size.

  Whether it is better to use records or multidimensional arrays in a problem is not always a clear-cut decision. Think of the problem of representing points in the $x - y$ plane discussed above. The obvious representation would be a record or structure such as:

```
struct point {
     int x, y;
};
```

instead of as a two-element array. A big plus for records is that the notation `p.y` and `p.y` are more akin to our natural notation for working with points. However, a

disadvantage of the record representation is that you cannot loop over individual variables as you can elements in an array.

Suppose you wanted to change a geometric program to work with three-dimensional points instead of two, or even in arbitrary dimensions. Sure you can easily add extra fields to the record, but every place where you did calculations on x and y you now must replicate them for z. But by using the array representation, changing distance computations from two to three dimensions can be as simple as changing a constant:

```
typedef int point[DIMENSION];

double distance(point a, point b)
{
        int i;
        double d=0.0;

        for (i=0; i<DIMENSION; i++)
                d = d + (a[i]-b[i]) * (a[i]-b[i]);

        return( sqrt(d) );
}
```

In Chapter 2, we will review more advanced data *structures* which can be built from these basic primitives. These will let us work with higher levels of abstraction, but don't be afraid to use simple technology when it suffices for the job.

1.5 About the Problems

Each chapter of this book ends with an appropriate set of programming challenge problems. These challenges have been carefully selected from over 1,000 such problems collected at the Universidad de Valladolid website. We aimed for clear, clever problems with varying degrees of difficulty. We especially looked for that spark of insight which turns a problem into a challenge.

The description of each selected problem has been edited for correctness and readability. We have tried to preserve the local color and flavor of each original problem while making the language reasonably consistent. The identification number for each problem on both judging sites is provided. These numbers are important for successful submission. The first number for each pair is associated with *http://www.programming-challenges.com*; the second number works on *http://online-judge.uva.es*.

To give some idea as to the relative difficulty of the problems, each has been annotated in three different ways. First, each problem has been given a grade of `A`, `B`, or `C`, reflecting how many correct solutions the judge has seen over the years. `A` problems are presumably easier to solve or somehow more attractive than `B` problems. Second, the frequency that submitted solutions for each problem are accepted by the judge is graded as `high`,

`average`, or `low`. Low percentages may indicate particularly finicky judges, or perhaps problems that require more subtlety than is initially apparent. Or they may just reflect bugs in test cases which have since been fixed. Therefore, do not obsess too much about these ratings. Finally, we give a subjective rating (from 1 to 4) of the academic level required to solve the problem. Higher numbers indicate more sophisticated problems.

Good luck, and happy hacking!